

# The Blueprint, Part 3: शिस्त आणि खऱ्या इमारती

---

August 30, 2026

# THE BLUEPRINT

## Book 5, Part 3: Elite शिस्त आणि खऱ्या इमारती

**मागच्या भागाचा शेवट:** आरतीला रचना कळली, वाचता आली. पण शेवटचा प्रश्न उरला: **elite टीम कशी काम करते?** 30-50 लोक एकमेकांच्या पायात न येता, चुका न करता, वेगात कसे बांधतात? हा भाग त्याचा: git, testing, review, incident-संस्कृती: आणि मग जगातल्या खऱ्या इमारती उलगडून (WhatsApp, UPI, Zerodha, Netflix). शेवटी: AI-युगात हे सगळं का उरतं, अंतिम परीक्षा, आणि नजरेचे checklists.

## सूत्र (शेवटच्या भागासाठी)

मोठेपणा फुकट नसतो.  
प्रत्येक रचना-निर्णय म्हणजे काहीतरी विकून काहीतरी विकत घेणे.  
Part 3 चा भर: रचना बांधणारी माणसं आणि त्यांची शिस्त.

## कपाटावर एक नजर

|        |               |                               |                      |
|--------|---------------|-------------------------------|----------------------|
| Book 1 | The Machine   | Book 2                        | The Thinking Machine |
| Book 3 | The Land Game | Book 4                        | The Harness          |
| Book 5 | The Blueprint | <- हे, Part 3: शिस्त + इमारती |                      |

## Part 2 ची उजळणी: 4 प्रश्न (बोलून; उत्तरं खाली)

1. नवें app: database डिफॉल्ट काय?

SQL: नाती+शुद्धता फुकट, नंतर सोडता येतात. NoSQL खरी गरज असेल तरच. Fashion नको.  
(2.2)

2. दोन सेवा कशा बोलाव्यात?

फक्त API ने (करार); एकमेकांच्या database ला थेट हात कधीच नाही. (2.5)

3. Monolith की microservices, सुरुवातीला?

Monolith; तुकडे तेव्हाच जेव्हा टीम-गर्दीचं दुखणं खरंच जाणवेल. Amazon उलट जाऊन 90% वाचवले. (2.6)

#### 4. Migration ची सुरक्षित पद्धत?

विस्तार-आकुंचन: छोटी, उलटवता-येणारी पावलं; backup आधी. (2.9)

## या भागात काय

Elite teams ची शिस्त (git, testing, CI/CD, debugging, blameless postmortem, canary), मग चार खऱ्या इमारती उलगडून, मग AI-युगाची रचना (Book 4 शी पूल), आणि अंतिम परीक्षा + design-नजरेचे checklists. रचना कळली; आता ती बांधणारी संस्कृती.

## विभाग 3: Elite Teams ची शिस्त

रचना कितीही चांगली असो, ती बांधणारी माणसं शिस्तीने काम करत नसतील तर सगळं कोसळतं. हा विभाग त्या शिस्तीचा: git, testing, review, deploy, आणि सर्वात महत्वाचं: चूक झाल्यावर काय करायचं (incident-संस्कृती). **हे तुमच्या धंद्यात कुठे येईल:** तुम्ही एकटे बांधाल किंवा agents सोबत (Book 4), पण ही शिस्त तीच: कारण ती “अनेक हात, एक दिशा, कमी चुका” ची शिस्त आहे: आणि agent हाही एक हात आहे. Elite teams वेगळे नसतात कारण ते हुशार असतात; ते वेगळे असतात कारण त्यांची **पद्धत** चुका पचवते.

## Chapter 3.1 [SPINE]: Git: इतिहास-यंत्र आणि Review-संस्कृती

Elite शिस्तीचा पाया एका साध्या पण क्रांतिकारी साधनात आहे: **git**. आरतीला 30 engineers सोबत काम करायचंय; त्या सगळ्यांचं code एकत्र, चुका न होता, कसं जमवायचं? git चं उत्तर दोन भागांत: इतिहास-यंत्र, आणि review-संस्कृती.

**भाग 1: git = time-machine.** Book 4 त हे थोडं भेटलं (4.5); आता पूर्ण. git प्रत्येक बदलाची नोंद ठेवतं: कोणी, कधी, काय, का बदललं: आणि कुठल्याही जुन्या क्षणी परत जाता येतं. लेखकाच्या draft-वहीची उपमा: प्रत्येक खाडाखोड जपलेली, म्हणून “तीन दिवसांपूर्वीचं वाक्य बरं होतं” वाटलं तर परत आणता येतं. Software मध्ये हे अमूल्य: एक बदल app फोडतो, git ने **लगेच** आधीच्या चालत्या क्षणी परत (Knight Capital ची \$460M-रात्र, 1.13: त्यांच्याकडे सुरक्षित परत-जाणं नव्हतं). “परत फिरता येणं” (Book 4, 4.5 चा brake) git देतं: म्हणून engineers **न घाबरता** प्रयोग करतात.

**भाग 2: अनेक हात, एक code (branches + merge).** 30 लोक एकाच code वर एकदम लिहिले तर गोंधळ (2.4 चा race, माणसांत!). git ची युक्ती: प्रत्येकाने आपली वेगळी **शाखा** (branch) घ्यायची: मुख्य code ला हात न लावता, स्वतःच्या प्रतीवर काम. काम झालं, तपासलं, की ती शाखा मुख्यात **जोडायची** (merge). प्रत्येक झाडाची वेगळी फांदी, नीट तपासून खोडाला जोडायची. आता 30 लोक समांतर काम करतात, एकमेकांच्या पायात न येता.

**भाग 3 (सर्वात महत्वाचा): review-संस्कृती (code review).** आणि इथे elite teams चं खरं रहस्य आहे. कुठलीही फांदी मुख्यात जोडण्याआधी, **दुसरा engineer ती वाचतो आणि मंजूर करतो** (pull request review). का? कारण: (1) दुसऱ्याची नजर तुमची चूक पकडते (तुम्हाला न दिसलेली); (2) ज्ञान पसरतं (दोघांना आता ते code कळतं, एकावर अवलंबित्व नाही); (3) दर्जा टिकतो (review पास होणारंच आत). Google, आणि जगातल्या प्रत्येक elite टीमचा लोखंडी नियम: **कुठलंही code एकट्याने मुख्यात जात नाही; दुसऱ्याची मोहोर लागतेच.** (Book 4, 5.3 चा “दुसऱ्याची झडती” इथे माणसांत; आणि AI-युगात reviewer कधी agent ही असू शकतो, पण मोहोरीचं तत्त्व तेच.)

**सूत्र इथे:** review = दर्जा + ज्ञान-प्रसार विकत, थोडा वेग विकून. एकट्याने थेट टाकणं जलद, पण चुका आणि एकाकी-ज्ञान. Review थोडं मंद, पण चुका आधीच पकडल्या, ज्ञान वाटलं. आणि हा सौदा elite teams नेहमी दर्जाच्या बाजूने करतात: कारण “थोडा उशीर” स्वस्त, “production मधली चूक” महाग.

## महाग चूक

“घाई आहे, review वगळून थेट टाकतो, नंतर बघू.” Elite teams मध्ये review वगळणं ही जवळपास गुन्हा-मानली-जाणारी गोष्ट, आणि कारण साधं: **review न झालेलं code हे न-तपासलेलं गृहीत** आहे, आणि ते बहुधा गर्दीच्या-महत्त्वाच्या क्षणी फुटतं. “घाई” हे review वगळायचं कारण नाही; उलट घाईच्या वेळी review **जास्त** महत्त्वाचं (घाईत चुका जास्त). एकट्याने-थेट- टाकणं आज 10 मिनिटं वाचवतं, उद्या production-bug चे 10 तास खातं. नियम: **मोहोर नेहमी, अपवाद नाही.**

## नकाशावर

Part 3 चा नकाशा (वाढत जाईल):

3.1 git: इतिहास-यंत्र (परत फिरता येणं) + branches (समांतर काम) + review (दुसऱ्याची मोहोर: दर्जा+ज्ञान-प्रसार)

**सूत्रावर:** elite शिस्तीची पहिली वीट: review चा सौदा (थोडा वेग देऊन दर्जा घेणं). माणसांची चूकही “अपयश गृहीत धरा” (Book 4) चा भाग: दुसऱ्याची नजर हा माणसातला brake.

## स्वतः बघा (5 मिनिटं)

तुम्ही लिहिलेलं काहीही (message, अर्ज) पाठवण्याआधी दुसऱ्याला “वाचून बघ” म्हटल्यावर त्यांनी चूक/सुधारणा दाखवल्याचं आठवा: तुम्हाला न दिसलेली. तीच code review ची ताकद: **तुमचे डोळे तुमच्या चुकीला सरावलेले असतात; दुसऱ्याचे नाहीत.** म्हणून elite teams एकमेकांचं सगळं वाचतात, न चुकता.

## विचार करा

आरतीच्या टीमने review सुरू केलं, चुका घटल्या. पण तरी काही चुका review मधून निसटतातच (माणूस थकतो, घाईत बघतो). आणि त्या production मध्ये पोहोचल्यावर गिऱ्हाईकाला दिसतात. आरती विचारते: “माणसाची नजर चुकतेच. पण एकच चूक पुन्हा- पुन्हा होऊ नये, आणि जुनं जमणारं नव्या बदलाने मोडू नये: याची **आपोआप** खात्री कशी करायची? दर वेळी 30 लोकांनी सगळं हाताने तपासणं अशक्य आहे.”

हवी ती आपोआप-तपासणी: आणि तिचं नाव testing. माणसाची नजर एकदाची; testing ची नजर दर वेळी, न थकता, न विसरता. Book 4 च्या evals चा (5.1) हा software-भाऊ. पुढचा chapter.

## Chapter 3.2 [SPINE]: Testing चे थर

आरतीच्या “आपोआप, न थकता तपासणी” या मागणीचं नाव testing. Book 4 त हे agent साठी evals म्हणून भेटलं (5.1); इथे ते software साठी, आणि त्याला वेगळे थर आहेत: प्रत्येक थर वेगळी चूक पकडतो.

**Testing म्हणजे काय.** code ची **आपोआप परीक्षा**: “हे code या-इनपुटला हे-उत्तर देतं का?” एकदा लिहिली की **दर वेळी** चालते: माणूस थकतो, विसरतो; test न थकता, न विसरता, प्रत्येक बदलावर तपासते. गुणवत्ता-नियंत्रकाची उपमा: कारखान्यात प्रत्येक माल तपासणारा, न कंटाळता, तोच निकष लावणारा. Testing हा elite teams चा तो नियंत्रक.

**तीन थर (प्रत्येक वेगळी चूक पकडतो):**

1. Unit test (सुटा भाग): एक छोटा तुकडा एकटा तपासणं.  
“भाव-मोजणीचं function 2 किलो x 40 = 80 देतं का?”  
जलद, भरपूर, स्वस्त. पाया.
2. Integration test (जोड): दोन-तीन भाग मिळून तपासणं.  
“ऑर्डर-सेवा + database + payment मिळून नीट चालतात का?”  
कमी, थोडे मंद, पण खरे-जगाजवळ.
3. End-to-end test (पूर्ण): गिऱ्हाईकासारखं पूर्ण प्रवास.  
“app उघडून, साखर शोधून, ऑर्डर करून, पैसे भरून: सगळं जमतं का?” सर्वात कमी, सर्वात मंद, सर्वात खरे.

**थरांचं तत्त्व (testing pyramid): खाली भरपूर, वर थोडे.** भरपूर unit tests (जलद-स्वस्त, रोज हजारो वेळा चालतात), मध्यम integration, थोडे end-to-end (मंद-महाग). का? कारण unit tests सेकंदात चालतात, end-to-end मिनिटं खातात: सगळंच end-to-end केलं तर testing इतकं मंद की कुणी चालवणारच नाही. पाया रुंद, कळस अरुंद: हीच शहाणी रचना.

**सर्वात मोठा फायदा: घसरण-रोध (regression).** आजची चूक दुरुस्त केली + तिची test लिहिली, तर उद्या कुणीतरी तेच पुन्हा मोडलं की **test लगेच ओरडते**: production आधी. Book 4, 5.5 चा “जुनं जमणारं नव्याने बिघडू नये” इथे शब्दशः. Elite teams चा नियम: **bug सापडला की आधी त्याची test लिहा (म्हणजे तो पुन्हा कधीच परत येणार नाही), मग दुरुस्त करा.** प्रत्येक चूक एकदाच; test कायमचा पहारा.

**सूत्र इथे:** testing = आत्मविश्वास + घसरण-रोध विकत, “आधीचा वेळ” विकून. Test लिहायला वेळ लागतो (आजचा खर्च); पण मग प्रत्येक बदल **न घाबरता** करता येतो (test राखतो: Book 4, 5.5 ची हिंमत). Test नसलेलं मोठं code म्हणजे प्रत्येक बदलाला भीती; test असलेलं म्हणजे धीट प्रयोग. वेग testing मुळे मरत नाही, वाढतो: फक्त सुरुवातीचा खर्च दिसतो, पुढची बचत नाही.

## महाग चूक

”Testing म्हणजे वेळ वाया; code तर चालतंय.” सर्वात महाग अल्पदृष्टी. Test नसलेलं code सुरुवातीला जलद वाटतं, पण लवकरच प्रत्येक बदल भीतीदायक होतो (“हे बदललं तर काय मोडेल?”), आणि टीम बदल करणं टाळते (2.11 चा “चालतंय ते मोडू नको” चा भाऊ): app गोठतं. उलट, test असलेलं code वर्षानुवर्ष धीटपणे बदलता येतं. Test हा “वेळ वाया” नाही, तो **भविष्यातल्या वेगाचा विमा** आहे. जगातलं प्रत्येक टिकाऊ मोठं software भरपूर testing वर उभं आहे: अपवाद नाही.

## नकाशावर

- 3.1 git: इतिहास + branches + review
- 3.2 testing: तीन थर (unit जलद-भरपूर / integration / e2e मंद-थोडे); घसरण-रोध; bug -> आधी test मग दुरुस्ती; वेगाचा विमा

**सूत्रावर:** testing चा सौदा वेळेत उलटतो: आजचा खर्च, उद्याची मुक्ती. Elite teams दीर्घ नजरेने हा सौदा करतात; नवखे आजच्या वेगाच्या मोहात तो टाळतात आणि उद्या गोठतात.

## स्वतः बघा (5 मिनिटं)

तुम्ही एखादं महत्वाचं गणित/हिशेब करताना “एकदा तपासून बघू” म्हणून उलटा हिशेब करता (80 / 40 = 2 किलो, बरोबर): ती एक मानसिक unit test आहे. आता कल्पना करा हे तपासणं **आपोआप, दर वेळी** झालं तर किती निश्चित. Software चं testing तेच, फक्त यंत्राने, हजारो वेळा, न थकता.

## विचार करा

आरतीच्या टीमकडे आता git, review, testing आहे. पण तरी एक अडचण: एका engineer ने बदल केला, review झालं, tests पास झाले: पण ते **गिन्हाईकांपर्यंत पोहोचवायला** (deploy) दर वेळी कुणीतरी हाताने, रात्री जागून, अनेक पायऱ्या करायचं: आणि तिथेच चुका (एक server विसरणं: Knight Capital!). “बदल तयार आहे, पण तो सुरक्षित, आपोआप, चुकल्याशिवाय गिन्हाईकांपर्यंत कसा न्यायचा?”

हवी ती आपोआप, सुरक्षित वाहतूक: build -> test -> deploy ची यंत्र-साखळी. तिचं नाव CI/CD, आणि ती elite teams च्या वेगाचं रहस्य आहे. पुढचा chapter.



## Chapter 3.3 [SPINE]: CI/CD आणि सुरक्षित Deploy

आरतीच्या “आपोआप, सुरक्षित वाहतूक” या मागणीचं नाव CI/CD. आणि त्यासोबत deploy सुरक्षित करण्याच्या युक्त्या: ज्यांच्या गैरहजेरीने Knight Capital आणि CrowdStrike कोसळले (1.13).

**CI/CD म्हणजे काय (साध्या भाषेत).** दोन जोडलेल्या साखळ्या: **CI (Continuous Integration):** कुणीही code बदललं की **आपोआप:** build (जोडणी) + सगळे tests चालवा (3.2) + review-दार (3.1). पास तरच पुढे. **CD (Continuous Deployment/Delivery):** पास झालेलं code **आपोआप, सुरक्षितपणे** गिन्हाईकांपर्यंत. कारखान्याच्या assembly-line ची उपमा: कच्चा माल (code-बदल) एका टोकाला; प्रत्येक टप्प्यावर आपोआप तपासणी-दार; बाहेर पडतं ते तयार, तपासलेलं उत्पादन. हाताने-रात्री-जागून चं काम यंत्राने, दर वेळी सारखं, चुकल्याशिवाय.

**पण deploy हा सर्वात धोक्याचा क्षण: तीन सुरक्षा-युक्त्या.** नवं code गिन्हाईकांपर्यंत नेणं म्हणजे चालत्या विमानाचं engine बदलणं: इथेच सर्वात मोठे अपघात (Knight, CrowdStrike). म्हणून elite teams तीन युक्त्या वापरतात:

1. Canary (कोळशाच्या खाणीतला पक्षी): नवं code आधी फक्त 1-2% गिन्हाईकांना द्या. काही बिघडलं का बघा. ठीक असेल तरच हळूहळू सगळ्यांना. CrowdStrike ने हे केलं नाही: 85 लाख यंत्रांना एकदम सदोष update = इतिहासातला सर्वात मोठा IT-बिघाड. एक canary त्यांना वाचवता आला असता.
2. Blue-green (दोन रंग): जुनं (blue) चालू ठेवून नवं (green) बाजूला तयार करा; तयार झाल्यावर वाहतूक हळूच green कडे वळवा. बिघडलं तर लगेच blue कडे परत (एका सेकंदात).
3. Feature flag (स्विच): नवं feature code मध्ये आहे पण "बंद" स्विचवर; हवं तेव्हा थोड्यांसाठी "चालू", बिघडलं तर कोड न बदलता लगेच "बंद". प्रयोग आणि सुरक्षा एकत्र.

तिन्हींचं एक तत्त्व (Book 4, 4.5 + 7.4 चे भाऊ, आणि 1.13 च्या प्रलयांचा थेट धडा): **कुठलाही बदल सगळ्यांना एकदम देऊ नका; आधी थोड्यांवर, परत-फिरण्याचा रस्ता उघडा ठेवून.** हेच elite deploy चं हृदय.

**आणि इथे एक थक्क करणारा विरोधाभास (elite teams चं खरं रहस्य):** जे teams जास्त वेळा deploy करतात (दिवसातून अनेकदा!) ते **कमी** अपघात करतात. कसं? कारण छोटे-वारंवार बदल सुरक्षित असतात (एक छोटा बदल बिघडला तर शोधणं-दुरुस्त करणं सोपं); मोठे-कचित बदल

धोकादायक (शेकडो बदल एकदम, काहीतरी फुटलं तर कशाने ते शोधणं अशक्य). Knight Capital चा मोठा-कचित deploy विरुद्ध elite चा छोटा-वारंवार. धडा: **वेग आणि सुरक्षा विरोधी नाहीत; छोटे-वारंवार-आपोआप बदल दोन्ही देतात.**

## महाग चूक

”Deploy म्हणजे मोठा प्रसंग: महिन्यातून एकदा, रात्री, सगळं एकदम, सगळे जागून.” हीच जुनी, धोकादायक पद्धत: मोठे-कचित- हाताने deploys = जास्त चुका, अवघड निदान (Knight ची रात्र). Elite उलट: **छोटे, वारंवार, आपोआप, canary सह.** “deploy भीतीदायक आहे” ही भावना म्हणजे तुमची deploy-पद्धत चुकीची आहे याचं लक्षण: सुरक्षित पद्धतीत deploy इतका सामान्य होतो की भीती उरत नाही. भीती हे सुधारणेचं आमंत्रण आहे, सहन करण्याची गोष्ट नाही.

## नकाशावर

3.1-3.2 git | testing

3.3 CI/CD: आपोआप build+test+deploy साखळी; canary (थोड्यांना आधी) + blue-green + feature flag; छोटे-वारंवार = सुरक्षित; वेग आणि सुरक्षा एकत्र

**सूत्रावर:** इथे सूत्राला सुखद उलथापालथ: छोटे-वारंवार deploys मध्ये वेग **आणि** सुरक्षा दोन्ही मिळतात: कारण जोखीम छोटी राहते. काही ठिकाणी शहाणी रचना सौद्याचं गणितच बदलते.

## स्वतः बघा (5 मिनिटं)

तुमच्या फोनवरची apps आपोआप, वारंवार update होतात ना (तुम्हाला कळतही नाही)? ते CI/CD. आणि कधीकधी एखादं नवं feature “काही जणांना दिसतं, काहींना नाही” (feature flag/canary): company आधी थोड्यांवर तपासतेय. तुम्ही रोज elite deploy- शिस्तीच्या दुसऱ्या बाजूला उभे असता, नकळत.

## विचार करा

आरतीकडे आता पूर्ण शिस्त आहे. पण एका रात्री, सगळं असूनही, app कोसळलंच: canary मधून निसटलेला एक bug. गिन्हाईकं संतापली. दुसऱ्या दिवशी टीम-मीटिंगमध्ये आरतीला मोह झाला: “कोणी केलं हे? कोणाची चूक?” पण तिला जाणवलं की दोषी शोधल्याने पुढच्या वेळी लोक चुका **लपवतील**, सांगणार नाहीत. “चूक झाली की काय करायचं: दोषी शोधायचा, की काहीतरी वेगळं?”

हाच elite teams आणि सामान्य teams मधला सर्वात मोठा फरक: चूक झाल्यावरची **संस्कृती**. दोषी शोधणं की रचना शिकणं. या एका निवडीवर टीम टिकते किंवा कुजते. पुढचा chapter: incident-संस्कृती.

## Chapter 3.4 [SPINE]: Incident-संस्कृती: दोष नको, धडा हवा

आरतीच्या “दोषी शोधायचा की काहीतरी वेगळं” या प्रश्नावर हा chapter: आणि हा पूर्ण Book 5 मधला सर्वात महत्त्वाचा असू शकतो, कारण तो tech चा नाही, **माणसांचा** आहे: आणि माणसंच तर रचना बांधतात.

**दोन संस्कृती, दोन भविष्यं.** चूक (incident) झाल्यावर टीम दोन रस्त्यांपैकी एक घेते: (1) **दोष-संस्कृती:** “कोणी केलं? त्याला जबाबदार धरा.” परिणाम: लोक घाबरतात, चुका **लपवतात**, जोखीम टाळतात, प्रयोग थांबतो, आणि सर्वात वाईट: तीच चूक पुन्हा होते (कारण कुणी उघड बोललंच नाही, म्हणून रचना शिकली नाही). (2) **blameless संस्कृती (दोष-विरहित):** “व्यक्ती नाही, **रचना** का फसली? या चुकीला कुठली रचनात्मक फट कारणीभूत, आणि ती कशी बुजवायची?” परिणाम: लोक उघड बोलतात, प्रत्येक चूक रचना बळकट करते, आणि तीच चूक पुन्हा होत नाही.

**Blameless postmortem (शव-विच्छेदन, दोषाशिवाय).** Elite teams प्रत्येक मोठ्या incident नंतर एक लिखित शव-विच्छेदन करतात: **काय घडलं, का घडलं (रचनेच्या पातळीवर), आणि पुन्हा होऊ नये म्हणून रचनेत काय बदललं:** कुणाचं नाव-दोष नाही. आठवा: 1.13 मधल्या Amazon, Facebook, CrowdStrike या सगळ्यांनी **जाहीर**, प्रामाणिक postmortems लिहिली: “आमच्या रचनेत ही फट होती, आम्ही ती अशी बुजवली”: engineer ला बळीचा बकरा न करता. आणि Book 4 (1.7, 5.2) ची तीच शिकवण: **चूक ही context आहे, शिकण्याचं इंधन: लपवायची नाही, वाचायची.**

**मूळ तत्त्व: चांगली माणसं वाईट रचनेत चुका करतात.** Knight Capital चा engineer मूर्ख नव्हता; रचनेत “एक server विसरणं शक्य” ही फट होती (1.13). GitLab चा engineer निष्काळजी नव्हता; रचनेत “चुकीच्या server वर rm चालवणं सोपं + backup न-तपासलेले” या फटी होत्या (1.2). **व्यक्तीला दोष देणं म्हणजे रचनेतली खरी फट न बघणं:** आणि मग ती फट पुढच्या (चांगल्या) माणसाकडूनही तीच चूक करवते. म्हणून प्रश्न कधीच “कोणी?” नाही, नेहमी “रचनेत काय, आणि कसं बदलू?” (Book 4, 4.1 ची Hashimoto-सवय इथे टीम-पातळीवर: चूक -> रचनेत कायमची दुरुस्ती).

**आरतीचा निर्णय:** तिने दोषी शोधला नाही. तिने विचारलं: “हा bug canary मधून का निसटला? आपली canary-तपासणी कुठे कमी पडली?” उत्तर: canary फक्त 10 मिनिटं चालवत होते, हा bug 30 मिनिटांनी दिसे. दुरुस्ती: canary-वेळ वाढवला + या प्रकारची test जोडली. एक चूक, एक रचनात्मक सुधारणा, शून्य दोष: आणि टीमने शिकलं की **चूक सांगणं सुरक्षित आहे:** म्हणून पुढे ते जास्त उघड, जास्त धीट झाले.

## महाग चूक

”जबाबदारी निश्चित केली पाहिजे; कोणीतरी उत्तरदायी हवं.” हे बरोबर **वाटतं** पण घातक: दोष-संस्कृती अल्पकाळात “शिस्त” वाटते, दीर्घकाळात **विष**: लोक चुका लपवतात (म्हणून त्या मोठ्या होऊन फुटतात), प्रयोग टाळतात (म्हणून नवं काही होत नाही), आणि उत्तम लोक सोडून जातात (भीतीच्या वातावरणातून). जबाबदारी **रचनेची** असावी, व्यक्तीची नाही. “उत्तरदायित्व” म्हणजे “रचना दुरुस्त करण्याची सामूहिक जबाबदारी”, “बळीचा बकरा शोधणं” नाही. ही एक सांस्कृतिक निवड टीम टिकवते किंवा कुजवते.

## नकाशावर

3.1-3.3 git | testing | CI/CD  
3.4 incident-संस्कृती: दोष नको (लपवणं, भीती), blameless postmortem हवं (रचना का फसली, कशी बदलू); चांगली माणसं वाईट रचनेत चुकतात; प्रश्न “कोणी?” नाही, “रचनेत काय?”

**सूत्रावर:** इथे सूत्र माणसांत उतरतं: दोष-मुक्त संस्कृती म्हणजे “अल्पकालीन-समाधान (बळी) विकून दीर्घकालीन-शिक्षण विकत घेणं.” Elite teams हा सौदा नेहमी शिक्षणाच्या बाजूने करतात.

## स्वतः बघा (5 मिनिटं)

तुमच्या घरात/कामात कधी चूक झाल्यावर “कोणी केलं” पेक्षा “पुन्हा होऊ नये म्हणून काय बदलू” विचारल्याचं आठवा: फरक जाणवला का? दोष दिल्यावर माणूस बचावात्मक होतो, लपवतो; रचना विचारल्यावर उघड, सहकार्यशील. हेच तत्त्व 50-माणसांच्या टीमपासून घरापर्यंत सारखं. आजपासून एका चुकीला “कोणी” ऐवजी “काय बदलू” विचारून बघा.

## विचार करा

आरतीची टीम आता खरोखर elite आहे: शिस्त, संस्कृती, सगळं. आणि तिला उत्सुकता वाटली: “आम्ही जे शिकलो: सोपेपणा, शिस्त, सौदे-नजर, दोष-मुक्त संस्कृती: हेच जगातल्या सर्वात मोठ्या, सर्वात यशस्वी companies वापरतात का? WhatsApp चे 50 लोक, UPI चे अब्जावधी व्यवहार, Zerodha ची 30-लोकांची टीम: यांच्या **आत** काय आहे? त्यांच्या रचना उलगडून बघता येतील का?”

हाच पुढचा विभाग, आणि Book 5 चा कळस: चार खऱ्या इमारती उलगडून. आणि तुम्हाला आश्चर्य वाटेल: त्या सगळ्यांचं रहस्य “भारी-गुंतागुंत” नसून **कट्टर सोपेपणा** आहे. विभाग 4: खऱ्या इमारती.

## विभाग 4: खऱ्या इमारती

सिद्धांत पुरे; आता प्रत्यक्ष इमारती. चार खऱ्या, जगप्रसिद्ध systems उलगाडून बघू: WhatsApp, UPI, Zerodha, Netflix. प्रत्येकातून Part 1-3 चे सौदे आणि शिस्त **प्रत्यक्ष** कसे वापरले ते दिसेल. **हे तुमच्या धंद्यात कुठे येईल:** हे नकाशा- वाचनाचे (2.10) खरे व्यायाम आहेत; आणि एक समान धागा दिसेल जो पूर्ण पुस्तकाचं सार आहे: **सर्वात यशस्वी systems सर्वात हुशार नसतात, सर्वात सोपी असतात.** आकडे तपासलेले.

## Chapter 4.1 [SPINE]: इमारत: WhatsApp: 50 माणसं, 90 कोटी लोक

पहिली इमारत, आणि सर्वात थक्क करणारी. 2015 त WhatsApp ~90 कोटी (900 million) लोकांना real-time chat देत होतं, अब्जावधी messages रोज: आणि engineering-टीम ~50 माणसं. (2014 त Facebook ने ~\$19 अब्जांना विकत घेतलं तेव्हा एकूण ~55 कर्मचारी, ~45 कोटी users: आकडे तपासलेले.) हे कसं? उत्तर पूर्ण पुस्तकाचं सार आहे.

**रहस्य 1: कट्टर सोपेपणा (feature नकार).** WhatsApp ने जे केलं नाही ते जास्त महत्वाचं: जाहिराती नाहीत, games नाहीत, फालतू features नाहीत, रंगीबेरंगी गुंतागुंत नाही. फक्त: संदेश पोहोचवा, विश्वासाने, जलद. एक काम, कट्टरपणे. कमी features = कमी code = कमी bugs = कमी माणसं. Book 4, 9.3 चा “एकट्याचा कारखाना” इथे 50-लोकांचा: सोपेपणा हाच सर्वात मोठा scaling- मंत्र.

**रहस्य 2: योग्य हत्यार (Erlang).** त्यांनी एक खास भाषा वापरली (Erlang): जी लाखो सतत-दुवे (2.8 चे WebSocket-सारखे) सांभाळण्यासाठीच दशकांपूर्वी (टेलिफोन-यंत्रणांसाठी) बनली होती. एका server वर ~20 लाख जोडण्या. योग्य कामाला योग्य हत्यार: “cool” भाषा नाही, या कामाला बनलेली भाषा (2.2 चा “fashion नको, सौदा बघा” इथे भाषेच्या निवडीत).

**रहस्य 3: नकाशा उलगडून (Part 1-2 चे सौदे).** WhatsApp मध्ये तुम्हाला दिसेल: (1) messages कायमचे साठवत नाहीत (पोहोचले की server वरून पुसतात): storage स्वस्त, गुंतागुंत कमी: एक धाडसी सौदा (2.1 चा data-model, टोकाला नेलेला). (2) real-time साठी सतत-दुवे (2.8). (3) प्रचंड-वेगवान, सैल लेखनासाठी योग्य database (LSM-कुळ, 2.3). प्रत्येक निर्णय “या कामाला काय लागतं” वर, “काय भारी दिसतं” वर नाही.

**समान धागा (पूर्ण पुस्तकाचा):** WhatsApp ने कुठलीही “जादूई भारी रचना” वापरली नाही. त्यांनी एक काम निवडलं, ते कट्टर सोपेपणाने केलं, योग्य हत्यार वापरलं, आणि प्रत्येक सौदा जाणून केला. म्हणून 50 माणसं 90 कोटींना पुरली. हा तुमच्यासाठी (आणि आरतीसाठी) सर्वात मोठा दिलासा: मोठेपणाला प्रचंड टीम/ पैसा/गुंतागुंत लागत नाही; सोपेपणा आणि योग्य निवडी लागतात. Book 4 चं leverage-सूत्र (9.3) इथे 2015 तच खरं ठरलं होतं.

### महाग चूक

“मोठं app म्हणजे भरपूर features, भरपूर माणसं, भरपूर गुंतागुंत.” WhatsApp हे उलट सिद्ध करतं: **features वाढवणं सोपं, नाकारणं अवघड:** आणि नाकारण्यातच खरी शिस्त. प्रत्येक जोडलेलं feature = जास्त code + bugs + माणसं + गुंतागुंत (Book 4, 4.2/4.4 चा “किती tools” तोच धडा). यशस्वी

होण्यासाठी जास्त करावं लागतं हा भ्रम; बहुधा **कमी, पण उत्तम** करावं लागतं. “काय करू” पेक्षा “काय करू नको” हा कठीण आणि किमती प्रश्न.

## नकाशावर

विभाग 3 (शिस्त): git | testing | CI/CD | incident-संस्कृती

4.1 WhatsApp: 50/90कोटी; कट्टर सोपेपणा (feature नकार) + योग्य हत्यार (Erlang) + जाणीवपूर्वक सौदे; मोठेपणा = सोपेपणा, प्रचंड-टीम नाही

**सूत्रावर:** WhatsApp पूर्ण पुस्तकाचं सूत्र जगून दाखवतं: प्रत्येक सौदा जाणून, सर्वात सोपी रचना निवडून, त्यांनी मोठेपणा थोड्या माणसांत पेलला. सोपेपणा हा सर्वात कमी लेखलेला superpower.

## स्वतः बघा (5 मिनिटं)

WhatsApp उघडा आणि मोजा: त्यात किती “features” आहेत? मुख्य: संदेश, call, status. संपलं जवळपास. आता एखादं गचडलेलं app आठवा (डझनभर features, गोंधळ). कुठलं तुम्ही रोज वापरता, कुठलं टिकलं? सोपेपणाची ताकद तुमच्या स्वतःच्या फोनवर दिसते.

## विचार करा

WhatsApp एका company चं. पण आरतीला उत्सुकता: “एका company ला 50 माणसं पुरली. पण **UPI** तर पूर्ण देशाची यंत्रणा: अनेक बँका, अब्जावधी व्यवहार, सगळे एकमेकांशी जोडलेले. एका company चं system आणि अख्ख्या देशाचं system: यात रचनेचा फरक काय? आणि इतक्या वेगवेगळ्या बँका एका जाळ्यात कशा बांधल्या?”

उत्तम प्रश्न: कारण UPI ही एका company ची इमारत नाही, तर **अनेकांना जोडणारा करार** (2.5 चा API, राष्ट्रीय पातळीवर!). UPI ची इमारत, आणि तिने जगाला काय शिकवलं: पुढचा chapter.



## Chapter 4.2 [SPINE]: इमारत: UPI: करार जो देश जोडतो

दुसरी इमारत, आणि सर्वात वेगळी: कारण UPI ही एका company ची नाही, तर **अख्ख्या देशाची** यंत्रणा. आणि तिचं रहस्य पूर्ण Part 2 च्या एका chapter मध्ये आहे: API म्हणजे करार (2.5).

**आकडे आधी (तपासलेले):** UPI जुलै 2026 त एका महिन्यात **~23-24 अब्ज (billion)** व्यवहार हाताळत होतं: दिवसाला ~76 कोटी. जगातली सर्वात मोठी real-time payment यंत्रणा, आणि ती **कुठल्याही एका company ची नाही.**

**रहस्य: UPI हे app नाही, करार आहे.** इथे 2.5 चा API-धडा राष्ट्रीय पातळीवर बघा. UPI च्या आधी: प्रत्येक बँकेचं वेगळं app, वेगळी यंत्रणा, एका बँकेतून दुसऱ्यात पैसे पाठवणं म्हणजे दिव्य. UPI ने काय केलं? एक **समान करार** (common API) बनवला: “पैसे पाठवायचे तर असं विचारा, असं उत्तर मिळेल”: आणि सगळ्या बँकांना त्या एका कराराशी जोडलं. आता Google Pay, PhonePe, Paytm, कुठलंही app, कुठलीही बँक: सगळे **एकाच कराराची भाषा** बोलतात. Book 4 चा MCP (4.4) आणि Book 5 चा API (2.5): दोन्हीचं हे राष्ट्रीय, भौतिक रूप. **पायाभूत करार सरकारने/सामायिक संस्थेने बनवला आणि खुला केला, म्हणून वरचा प्रचंड बाजार बहरला.**

**नकाशा उलगडून (Part 1 चे सौदे, देश-पातळीवर):** UPI मध्ये तुम्हाला दिसेल: (1) **प्रचंड वाचन-लेखन:** म्हणून sharding (1.6: व्यवहार बँक/प्रदेशानुसार वाटलेले), replication (1.5), load balancing (1.8): सगळं देश-आकारात. (2) **पैसा = कडक CAP-C** (1.10): फूट पडली तर “थांबा, पुन्हा प्रयत्न करा” (तुम्ही अनुभवलयं!): चुकीचा व्यवहार कधीच नाही: बँकिंग नेहमी अचूकतेच्या बाजूने. (3) **idempotency** (1.7): तोच व्यवहार दोनदा गेला तरी पैसे एकदाच कापले (म्हणून तुम्ही निर्धास्त refresh करता). प्रत्येक सौदा या पुस्तकातलाच, पण अब्जावधींच्या प्रमाणात.

**UPI ची outage-कथा (प्रामाणिक, तपासलेली):** एप्रिल 2025 त UPI ला मोठा बिघाड झाला (काही तास). कारण? बँकांच्या apps कडून “व्यवहार झाला का?” या तपासणी-विनंत्यांचा **पूर** (काही जुने व्यवहार पुन्हा-पुन्हा तपासत होते): यंत्रणा गुदमरली. उपाय (1.11 चा धडा प्रत्यक्ष): NPCI ने या तपासणी-विनंत्यांवर **मर्यादा** घातली (rate limiting). धडा: अब्जावधींची यंत्रणाही retry-पुराने (1.11: बेभान retry = आगीत तेल) कोसळू शकते; आणि उपाय तोच जो आपण शिकलो: मर्यादा, throttle.

## महाग चूक

”मोठं यश म्हणजे एका company ने सगळं स्वतः बांधणं, मालकी ठेवणं.” UPI उलट शिकवतं: **सर्वात मोठा प्रभाव खुल्या कराराने येतो, बंद मालकीने नाही.** जर प्रत्येक बँकेने स्वतःचं बंद app ठेवलं असतं, तर आजचं UPI नसतं. करार खुला केला, मालकी सोडली, म्हणून सगळे जोडले गेले आणि प्रचंड मूल्य निर्माण झालं (Book 4, 2.6/4.4 चा “पायाभूत गोष्ट फुकट, वरचा बाजार जिंक” तोच धडा). कधीकधी सर्वात मोठं जिंकणं म्हणजे **नियंत्रण सोडणं.**

## नकाशावर

विभाग 3 (शिस्त): `git...incident`  
 4.1 WhatsApp: सोपेपणा + योग्य हत्यार  
 4.2 UPI: करार जो देश जोडतो (API राष्ट्रीय); खुला करार >  
 बंद मालकी; पैसा=CAP-C, idempotency, sharding देश-आकारात;  
 retry-पुराने अज्जावधींची यंत्रणाही कोसळते (rate limit)

**सूत्रावर:** UPI दाखवतं की सर्वात मोठा सौदा कधी “मालकी विकून पोहोच विकत घेणं” असतो: खुला करार म्हणजे नियंत्रण सोडून प्रचंड-मूल्य घेणं. सौद्याचं सर्वात उदार रूप.

## स्वतः बघा (5 मिनिटं)

पुढच्या वेळी UPI ने पैसे पाठवताना विचार करा: तुमचं app (PhonePe/GPay), समोरच्याचं वेगळं app, दोन वेगळ्या बँका: आणि तरी सेकंदात पैसे पोहोचतात. हे शक्य आहे कारण सगळे **एकाच कराराची भाषा** बोलतात. तुम्ही रोज 2.5 चा API-धडा अज्जावधींच्या प्रमाणात वापरता, नकळत.

## विचार करा

WhatsApp खासगी company, UPI राष्ट्रीय यंत्रणा: दोन टोकं. आरतीला आता एक वेगळाच प्रश्न: “या दोन्हींनी वेगवेगळ्या रचना वापरल्या. पण एक भारतीय company जी शेअर-बाजाराचं काम करते: Zerodha: ती फक्त **30 लोकांच्या** tech-टीमने भारतातली सर्वात मोठी broker झाली, म्हणे. आणि त्यांचं तत्त्व ‘कंटाळवाणं tech वापरा’ असं आहे: नवीन-भारी काहीच नाही. हे कसं जमतं? सगळे तर नवीन tech च्या मागे धावतात.”

Zerodha ची गोष्ट पूर्ण पुस्तकाच्या सूत्राचा कळस आहे: “कंटाळवाणं tech जिंकतं” हे विधान तुम्हाला आता खोलवर पटेल. पुढचा chapter.

## Chapter 4.3 [SPINE]: इमारत: Zerodha: कंटाळवाणं Tech जिंकतं

तिसरी इमारत, आणि भारतीय अभिमानाची: Zerodha. भारतातली सर्वात मोठी stock-broker, कोट्यवधी गिन्हाईक, अब्जावधींचे रोजचे व्यवहार: आणि tech-टीम फक्त ~30 माणसं (CTO कैलाश नाथ). त्यांचं तत्त्व पूर्ण पुस्तकाच्या सूत्राचा कळस आहे, आणि ते धक्कादायक वाटतं: **”कंटाळवाणं tech वापरा.”**

**”Boring technology” म्हणजे काय.** सगळे नवीन, चकचकीत, “cool” tech च्या मागे धावतात (नवी database, नवी भाषा, नवी fashion). Zerodha उलट: **जुनं, सिद्ध, कंटाळवाणं, पण भरवशाचं** tech वापरा. त्यांची मुख्य database? PostgreSQL (जुनी, सिद्ध: 2.2). नवीन-चमकदार NoSQL नाही, microservices चा डोंगर नाही: बहुतांश एकसंध, साधी रचना (2.6 चा monolith, अभिमानाने). का? कारण **कंटाळवाणं tech = माहीत असलेले सौदे, कमी आश्चर्य, कमी रात्री-जागरणं.**

**यामागचं खोल शहाणपण (सूत्राशी थेट):** नवीन tech म्हणजे **न-माहीत सौदे:** ते कुठे फुटेल, कसं वागेल, कुणालाच पूर्ण माहीत नाही (कारण नवीन). जुनं, कंटाळवाणं tech म्हणजे **हजारो लोकांनी वर्षानुवर्ष वापरून सगळे सौदे-धोके उघड केलेले:** तुम्हाला आश्चर्य वाटणार नाही. Zerodha पैशाचं काम करते (2.2: पैसा = कडक शुद्धता): तिथे “आश्चर्य” म्हणजे गिन्हाईकाचे पैसे गेले. म्हणून ते जाणीवपूर्वक **कंटाळवाणं-पण-भरवशाचं** निवडतात. Book 2 च्या money-map ची भाषा: **दुर्मिळता आणि भरवसा यांना भाव आहे; नवेपणाला नाही.**

**अजून एक Zerodha-वैशिष्ट्य: कमी माणसं, कमी घाई.** ~30 लोकांची टीम, “deadline नसलेली” संस्कृती (त्यांच्या स्वतःच्या लेखनातून), आणि tech-debt (जुनं-गचडलेलं code) आक्रमकपणे साफ करणं. आणि ते बरंचसं **FOSS** (मुक्त-स्रोत) जगाला परत देतात. WhatsApp चा सोपेपणा-मंत्र (4.1) इथे भारतीय broker- रूपात: **कमी माणसं, सोपं-सिद्ध tech, घाई नाही, दर्जा आधी.**

**हे तुमच्यासाठी सर्वात महत्त्वाचं का:** कारण हे तुमच्या मार्गाशी थेट जुळतं. तुम्हाला Google/Netflix ची अब्ज-डॉलर रचना बांधायची नाही; तुम्हाला **भरवशाचं, टिकाऊ, थोड्या माणसांत चालणारं** काहीतरी बांधायचंय. आणि त्यासाठी Zerodha हा आदर्श: नवीन- भारी tech चा मोह टाळा, सिद्ध-कंटाळवाणं निवडा, सोपं ठेवा, दर्जावर भर. **”कंटाळवाणं” हा अपमान नाही; तो प्रौढतेचा दागिना आहे.**

## महाग चूक

”आम्ही आधुनिक आहोत, नवीनतम tech वापरतो.” नवीन tech वापरणं हे कौशल्याचं लक्षण **वाटतं**, पण अनेकदा ते अनुभवाचं लक्षण असतं: अनुभवी engineer ला माहीत असतं की नवीन tech = न-माहीत सौदे = रात्री-जागरणं. “आम्ही latest वापरतो” चा अभिमान अनेक startups ना महागात पडला: fashion-ग्रस्त निवडींनी अस्थिर, अवघड-देखभाल systems, गिऱ्हाईकांचे गेलेले पैसे. Zerodha उलट: कंटाळवाणं, स्थिर, म्हणून सर्वात मोठं. **नवेपणाच्या मोहाला “हे या समस्येला खरंच लागतं का, की फक्त cool आहे” ने तपासा:** पूर्ण पुस्तकाचा “fashion नको, सौदा बघा” इथे शिखरावर.

## नकाशावर

विभाग 3 (शिस्त): git...incident

4.1 WhatsApp: सोपेपणा 4.2 UPI: खुला करार

4.3 Zerodha: कंटाळवाणं tech जिकतं (~30 लोक); जुनं-सिद्ध = माहीत सौदे, कमी आश्चर्य; पैशाला भरवसा > नवेपणा; कमी माणसं, घाई नाही, दर्जा आधी

**सूत्रावर:** Zerodha सूत्राचा सर्वात प्रौढ अर्थ दाखवतं: सर्वात शहाणा सौदा कधी “नवेपणाचा मोह विकून भरवसा विकत घेणं” असतो. Boring म्हणजे known trade-offs; known म्हणजे शांत झोप.

## स्वतः बघा (5 मिनिटं)

तुमच्या आयुष्यातलं एक “कंटाळवाणं पण भरवशाचं” साधन आठवा (जुनी सायकल, जुना फोन जो कधीच बंद पडत नाही). आणि एक “चकचकीत पण त्रासदायक” (नवं gadget जे सतत बिघडतं). महत्त्वाच्या, रोजच्या कामाला तुम्ही कुठलं निवडता? Zerodha पैशाच्या कामाला तेच निवडतं: कंटाळवाणं-भरवशाचं. प्रौढ निवड नेहमी भरवशाच्या बाजूने.

## विचार करा

WhatsApp सोपं, UPI खुलं, Zerodha कंटाळवाणं: तिन्हींनी सोपेपणा-भरवसा निवडला. पण आरतीला शेवटचा प्रश्न: “हे सगळे ‘अपयश टाळायचा’ प्रयत्न करतात. पण Netflix बदल ऐकलंय की ते **मुद्दाम स्वतःचे servers पाडतात**: ‘Chaos Monkey’ नावाचं काहीतरी जे यादृच्छिकपणे यंत्रं बंद करतं. अपयश टाळायचं सोडून मुद्दाम अपयश घडवायचं? हे वेडेपणा नाही का?”

वेडेपणा नाही, तर सर्वात प्रगल्भ शहाणपण: आणि तो Book 4 च्या red-teaming (7.4) चा infrastructure-भाऊ आहे. मुद्दाम-अपयशातून टिकाव कसा येतो: शेवटची इमारत, Netflix.

## Chapter 4.4 [SPINE]: इमारत: Netflix: मुद्दाम अपयश घडवणारे

शेवटची इमारत, आणि सर्वात विरोधाभासी: Netflix मुद्दाम स्वतःचे servers पाडतं. आरतीला हे वेडेपणा वाटलं; प्रत्यक्षात हे Book 5 च्या सर्वात प्रगल्भ धड्याचं मूर्त रूप आहे: **अपयश गृहीत धरा (1.11), आणि ते खरं होण्याआधी स्वतः घडवून बघा.**

**Chaos Monkey ची गोष्ट (तपासलेली).** ~2010-11 च्या सुमारास Netflix cloud कडे स्थलांतरित होत होतं (एका जुन्या database- बिघाडाने त्यांना धडा शिकवला होता). त्यांनी एक विचित्र यंत्र बनवलं: **Chaos Monkey**: जे कामाच्या वेळेत, यादृच्छिकपणे, त्यांचेच चालू servers **बंद पाडतं**. मुद्दाम. रोज. का हा वेडेपणा?

**तर्क सुंदर आहे:** “अपयश होणारच (1.11: मोठं system कधीच पूर्ण चालू नसतं). मग ते **आम्ही निवडलेल्या वेळी, आमच्या देखरेखीखाली** घडावं: की रात्री 3 वाजता, गिन्हाईकांच्या गर्दीत, आम्ही झोपेत असताना घडावं?” Chaos Monkey रोज servers पाडतं, म्हणून Netflix ला **भाग पडतं** प्रत्येक भाग “एक server पडला तरी चालला पाहिजे” असा बांधायला (1.8 चं stateless + load balancer, कट्टरपणे). जो भाग एका server-मृत्यूने कोसळतो तो Chaos Monkey **लगेच** उघड करतो: गिन्हाईकांच्या आधी. अग्निशमन-दलाची कवायत रोज करण्यासारखं: खरी आग लागल्यावर गोंधळ नको.

**हे Book 4 शी थेट जुळतं (red-teaming, 7.4):** Book 4 त आपण agent वर स्वतः हल्ला करायला शिकलो (शत्रूच्या आधी फट सापडावी). Chaos Monkey तेच infrastructure साठी: **स्वतःच्या system वर स्वतः “हल्ला” (अपयश) करून, दुर्बल जागा शत्रू (खरा बिघाड) येण्याआधी शोधणं.** दोन्हीचं एक तत्त्व: टिकाव हा गृहीत धरून येत नाही; तो सतत तपासून, मुद्दाम ताण देऊन कमवावा लागतो. याला “chaos engineering” म्हणतात, आणि ती आता elite teams ची प्रौढ शिस्त आहे.

**Netflix चे इतर धडे (नकाशा उलगडून):** (1) प्रचंड microservices (2.6): पण त्यांच्याकडे ते **खरंच** लागतात (प्रचंड टीम, जगभर scale): fashion नाही, गरज. (2) CDN चा प्रचंड वापर (1.9: video जगभर, जवळून: Netflix च्या रहदारीचा मोठा भाग). (3) “एक भाग पडला तरी अनुभव चालू राहील” ची कट्टर रचना (graceful degradation): शिफारसी नाही आल्या तरी video चालतो. प्रत्येक निर्णय “अपयश होणारच” गृहीत धरून.

**पूर्ण विभागाचा समान धागा (चार इमारती एकत्र):** WhatsApp (सोपेपणा), UPI (खुला करार), Zerodha (कंटाळवाणं-भरवशाचं), Netflix (मुद्दाम-अपयश). वरवर वेगळे, पण एक सूर: **कुणीही “जादूई हुशारी” वर अवलंबून नाही; सगळे रचनेच्या शिस्तीवर, जाणीवपूर्वक सौद्यांवर, आणि अपयश-गृहीत-धरण्यावर उभे.** हेच पूर्ण Book 5 चं सार: **मोठं software हुशारीने नाही, शिस्तीने आणि सौद्यांच्या नजरेने बांधलं जातं.**

## महाग चूक

”आमचं system चालतंय, त्याला हात लावायचा कशाला (मुद्दाम पाडणं तर दूरच).” “चालतंय” ही सर्वात धोकादायक शांतता: कारण तुम्हाला माहीत नाही ते का चालतंय, आणि कधी थांबेल. Netflix चा धडा: **जे तुम्ही तपासलं नाही ते तुम्हाला माहीत नाही**. अपयश-सज्जता “कधीतरी बघू” वर सोडली की ती नेमकी सर्वात वाईट क्षणी (खऱ्या बिघाडात) तपासली जाते: सर्वात महाग परीक्षा. मुद्दाम, नियंत्रित अपयश आजच घडवणं म्हणजे उद्याचा अनियंत्रित प्रलय टाळणं. शांततेवर विसंबू नका; ताण देऊन तपासा.

## नकाशावर

विभाग 3 (शिस्त): git...incident  
 4.1-4.3 WhatsApp (सोपेपणा) | UPI (खुला करार) | Zerodha (कंटाळवाणं)  
 4.4 Netflix: Chaos Monkey (मुद्दाम अपयश) = red-teaming चा infra-भाऊ; टिकाव तपासून कमवायचा; graceful degradation; चार इमारतींचा सूर: शिस्त+सौदे > हुशारी

**सूत्रावर:** Netflix सूत्राला त्याच्या धाडसी टोकाला नेतं: टिकाव विकत घ्यायचा तर “आजची शांतता” विकानी लागते (मुद्दाम ताण देऊन). सर्वात प्रगल्भ सौदा: नियंत्रित वेदना विकत, अनियंत्रित प्रलय टाळून.

## स्वतः बघा (5 मिनिटं)

अग्निशमन-कवायतीचा (fire drill) विचार करा: शाळा/office मध्ये मुद्दाम “आग लागली” म्हणून सराव. वेडेपणा वाटतो? नाही: खरी आग लागल्यावर गोंधळ नको म्हणून. Chaos Monkey तेच, servers साठी. आणि तुमच्या स्वतःच्या तयारीतही: महत्त्वाच्या गोष्टीचा backup **तपासून** बघितलात का, की “असेल” म्हणून सोडलं? (GitLab ची रात्र, 1.2.) मुद्दाम तपासणं ही प्रगल्भता.

## विचार करा

चार इमारती बघून झाल्या, सगळी शिस्त कळली. आरतीला आता एक अस्वस्थ करणारा प्रश्न: “हे सगळं मी शिकले: रचना, सौदे, शिस्त, इमारती. पण Book 4 म्हणालं की आता **agents code लिहितात**. मग हे सगळं शिकण्याचा उपयोग काय? जर यंत्रच बांधणार, तर माझ्या या ज्ञानाला किंमत उरते का? की हे सगळं आता जुनं झालं?”

हा सर्वात महत्त्वाचा प्रश्न, आणि त्याचं उत्तर तुमचं भविष्य ठरवतं. उत्तर छोटं: agents code लिहितील, पण **रचना बरोबर आहे का, सौदे योग्य आहेत का** हे ठरवणारी नजर तुमचीच: आणि तीच आता सर्वात मौल्यवान. AI-युगाची रचना: पुढचा chapter.

## Chapter 4.5 [SPINE]: AI-युगाची रचना: नजर हीच leverage

आरतीचा अस्वस्थ प्रश्न: agents code लिहितात, मग हे शिकण्याचा उपयोग काय? हा chapter त्याचं उत्तर आहे, आणि तो Book 4 शी थेट पूल बांधतो. उत्तर एका ओळीत: **code लिहिणं स्वस्त झालं; रचना बरोबर आहे का ठरवणारी नजर महाग राहिली: आणि तीच आता तुमची सर्वात मोठी संपत्ती.**

**काय बदललं, काय नाही.** Book 4 (9.2-9.3) चा leverage- सरकण्याचा धडा इथे software-रचनेत उतरतो. बदललं: **code टाईप करणं** (agents करतात, जलद-स्वस्त). **नाही** बदललं: “कुठली रचना, कुठला सौदा, कुठली database, हे बरोबर आहे का” हा निर्णय. आठवा (2.10): top engineer ची अर्धी किंमत नवं बांधण्यात, अर्धी **रचना वाचून सौदे-धोके ओळखण्यात**. आता बांधणं agent करतो; म्हणून **वाचणारी-ठरवणारी नजर** ही उरलेली, आणि म्हणून सर्वात मौल्यवान. Code = कच्चा माल (स्वस्त); नजर = मूल्य (Book 4, 9.1 चा थर 5-6).

**agent-सोबत बांधण्याची नवी शिस्त (Book 4 + Book 5 एकत्र):**

1. स्पष्ट करार आधी (Book 4, 4.7 spec-driven): agent ला “काय बांधायचं + कुठले सौदे” स्पष्ट सांगा. “किराणा-app बांध” नाही; “SQL वापर (पैसे आहेत), monolith ठेव (टीम छोटी), हे API-करार” : तुमची Book 5 ची नजर agent ला दिशा.
2. रचना तुम्ही ठरवा, code agent लिहो: database-निवड (2.2), monolith/micro (2.6), CAP (1.10): हे निर्णय तुमचे; अंमलबजावणी agent ची.
3. नकाशा वाचा (2.10): agent ने बांधलेलं वाचून सौदे-धोके तपासा: “इथे cache आहे, ‘जुनं कधी’ विचारलं का? इथे थेट database-हात आहे का (2.5 चा भंग)?”
4. Book 4 ची सगळी शिस्त लागू: evals (test), brakes, review (agent चं code ही reviewer ला/तुम्हाला दाखवा), observability.

**म्हणजे Book 5 शिकल्याने काय मिळालं:** agent ला दिशा देण्याची आणि त्याचं काम तपासण्याची नजर. Agent ला system-design कळत नाही असं नाही; पण **तुमच्या** परिस्थितीला कुठला सौदा योग्य (छोटी टीम? पैसे? किती गर्दी?) हे **तुम्ही** ठरवता: agent अंमलात आणतो. रचना न कळणारा माणूस agent ला “किराणा-app बांध” म्हणतो आणि agent जे देईल ते घेतो (बहुधा over-engineered, चुकीचे सौदे); रचना कळणारा माणूस agent ला नेमकी दिशा देतो आणि त्याचं काम तपासतो. **तोच फरक थर-4 (wrapper) आणि थर-5-6 (खंदक) मधला** (Book 4, 9.1).



**पूर्ण दोन पुस्तकांचं लग्न:** Book 4 ने “AI कसं वापरून बांधायचं” दिलं (harness); Book 5 ने “काय बांधायचं, कुठली रचना” दिलं (blueprint). एकत्र: **तुम्ही agent ला योग्य गोष्ट, योग्य रचनेत, सुरक्षितपणे बांधायला लावू शकता: आणि त्याचं काम तपासू शकता.** हीच 2026 ची सर्वात मौल्यवान जोडी: आणि आता ती तुमच्याकडे आहे.

## महाग चूक

”AI सगळं बांधतं, आता system-design शिकायची गरज नाही.” सर्वात महाग गैरसमज. उलट: **जितकं बांधणं स्वस्त, तितकी ‘काय-आणि-कशी बांधायची’ ची नजर मौल्यवान.** Agent ला रचना कळते, पण **तुमच्या** संदर्भातले सौदे (टीम-आकार, पैसा, गर्दी, data-स्वभाव) तुम्हीच द्यायचे; आणि त्याचं काम **तपासायची** नजरही तुमचीच. रचना न कळणारा agent-वापरकर्ता म्हणजे आंधळा माणूस driverless गाडीत: गाडी चालवेल, पण **कुठे जायचं** माहीत नसेल तर चुकीच्या जागी वेगात पोहोचेल. नजर हीच steering.

## नकाशावर

विभाग 3-4: शिस्त + चार इमारती

4.5 AI-युगाची रचना: code स्वस्त, नजर महाग; agent ला दिशा (तुमचे सौदे) + तपासणी (नकाशा वाचा) तुमची; Book 4 (कसं) + Book 5 (काय) = मौल्यवान जोडी

**सूत्रावर:** शेवटचा, सर्वात मोठा सौदा-उलगाडा: जेव्हा बांधणं फुकट होतं, तेव्हा **काय बांधायचं आणि कुठला सौदा** हा निर्णय सर्व मूल्याचा होतो. तुमची रचना-नजर हीच तुमची न-नकलता-येणारी संपत्ती.

## स्वतः बघा (5 मिनिटं)

कुठल्याही AI ला “एक किराणा-app ची रचना सुचव” विचारा. उत्तर वाचा आणि 2.10 ची सहा-प्रश्नांची नजर लावा: त्याने कुठली database सुचवली, का? Monolith की micro? Cache कुठे? आणि विचारा: “माझी टीम 2 माणसांची आहे, हे बदलतं का?” बघा उत्तर कसं बदलतं. तुम्ही agent ला **दिशा देताय आणि तपासताय:** हीच AI-युगातली तुमची भूमिका, प्रत्यक्ष.

## विचार करा

Book 5 संपत आलय. आरतीचा प्रवास पूर्ण: एक laptop पासून जगभर पसरलेल्या system पर्यंत, आणि आता agents ला दिशा देणाऱ्या नजरेपर्यंत. शेवटचा प्रश्न, जो पुस्तकाचं सगळं एका परीक्षेत बांधेल: “मी हे सगळं शिकले: नऊ सौदे, मोठे निर्णय, elite शिस्त, खऱ्या इमारती, AI-युगाची नजर. पण हे खरंच **मुरलं** का? की फक्त वाचून गेलं? हे मला कसं कळणार?”

तेच अंतिम परीक्षा ठरवेल: दोन पुस्तकांच्या (5, आणि Book 4) रचना-नजरेची. आणि मग पाच checklists, जे तुम्ही आयुष्यभर टेबलावर ठेवाल. Book 5 चा शेवट, पुढे.

## Chapter 4.6 [SPINE]: अंतिम परीक्षा

नियम आधीसारखेच: पुस्तक न उघडता, **बोलून** उत्तर द्या; गाभा खाली, आधी स्वतःचं बांधा मग ताडा. हेतू गुण नाही: **रचना-नजर मुरली का** हे बघणं.

### भाग अ: सौदे मुरले का (5 प्रश्न)

1. App हळू झालं. तुमचं पहिलं पाऊल, आणि “मोठा server घेऊ” का नाही?

रचना तपासा: गहाळ index (1.3)? गहाळ cache (1.4)? वाचनं replicas कडे वळवली का (1.5)? 90% हळूपणा रचनेत, ताकदीत नाही. पैसे ओतणं शेवटचा उपाय.

2. “आमचं नवं app सगळं NoSQL + microservices वर!” : तुमचे प्रश्न?

डेटाला नाती-शुद्धता लागते का (मग SQL: 2.2)? टीम एवढी मोठी आहे का की microservices चं टीम-स्वातंत्र्य खरंच लागतं (2.6)? की हे fashion आहे? Amazon उलट गेलं (-90%).

3. पैसे-व्यवहाराला CAP मध्ये C की A; आणि queue वापरली तर आणखी काय हवं?

C (फूट पडली तर थांबा, पण अचूक: 1.10). Queue सोबत idempotency हवीच (दोनदा-कापले-जाणं टाळायला: 1.7).

4. चालत्या, लाखो-नोंदींच्या database ची रचना बदलायची: पद्धत?

विस्तार-आकुंचन: नवं जोडा (जुनं ठेवून) -> दुहेरी-लेखन -> वळण -> जुनं काढा; छोटी उलटवता-येणारी पावलं; backup तपासलेला आधी (2.9, GitLab).

5. System निरोगी आहे हे कशाने ठरवाल, सरासरीने का नाही?

टोक (p99): सरासरी अल्पसंख्य-दुःखी लपवते, आणि तोच गिन्हाईक धंदा ठरवतो (1.12). आणि दिसण्यासाठी logs+traces (5.4-Book4/1.12).

### भाग ब: शिस्त, इमारती, AI-युग (5 प्रश्न)

6. चूक झाली. “कोणी केलं” का विचारू नये?

दोष-संस्कृती लोकांना चुका लपवायला लावते; blameless postmortem विचारतो “रचनेत काय फट, कशी बुजवू” (3.4). चांगली माणसं वाईट रचनेत चुकतात.

7. WhatsApp/Zerodha चा समान धडा एका ओळीत?

मोठेपणा हुशारीने नाही, **सोपेपणा + योग्य निवडी + जाणीवपूर्वक सौदे** यांनी: 50 माणसं/90कोटी, ~30 लोक/सर्वात मोठा broker (4.1, 4.3). कंटाळवाणं-सोपं जिकतं.

8. Netflix मुद्दाम servers का पाडतं, आणि हे Book 4 शी कसं जुळतं?

अपयश होणारच; ते आपल्या-वेळी-देखरेखीखाली घडवून दुर्बल जागा आधी शोधणं (chaos engineering: 4.4). हे red-teaming (Book 4, 7.4) चा infra-भाऊ: टिकाव तपासून कमवायचा.

9. Deploy सुरक्षित करायच्या तीन युक्ती, आणि CrowdStrike काय शिकवतं?

Canary (थोड्यांना आधी) + blue-green + feature flag (3.3). CrowdStrike ने 85 लाख यंत्रांना एकदम सदोष update दिलं: canary त्यांना वाचवता आला असता (1.13).

10. Agents code लिहितात, मग system-design शिकण्याचा उपयोग काय?

Code स्वस्त झालं; **रचना बरोबर आहे का, कुठला सौदा योग्य** ही नजर महाग राहिली (4.5). तुम्ही agent ला दिशा (तुमचे सौदे) देता आणि त्याचं काम तपासता (नकाशा वाचा). नजर = leverage (Book 4, 9.2).

आणि आता **खरी** अंतिम परीक्षा: वरचे दहा “जमले का” नाही: **खालचा एक**, ज्याचं उत्तर पुस्तकात नाही.

पूर्ण Book 5 एका वाक्यात बांधता येतं. आधी स्वतः लिहा, मग खाली बघा:

”सर्वात चांगलं software असं काही नसतं; फक्त **या गर्दीला, या माहितीला, या टीमला, या क्षणी योग्य** असतं: आणि प्रत्येक रचना-निर्णय म्हणजे काहीतरी विकून काहीतरी घेणं. जो हा सौदा **जाणून** करतो तो engineer; जो fashion बघतो तो दोनदा बांधतो. आणि agents च्या युगात, हा सौदा जाणण्याची नजर हीच माणसाची उरलेली, आणि सर्वात मौल्यवान, संपत्ती आहे.”

चार इमारती हेच जगल्या: WhatsApp चा सोपेपणा, UPI चा खुला करार, Zerodha चं कंटाळवाणं भरवसा, Netflix चं मुद्दाम-अपयश: कुणीही “जादूई हुशारी” वर नाही, सगळे **सौद्यांच्या नजरेवर** उभे.

## शेवटचं “स्वतः बघा” (आयुष्यभराचं)

आजपासून, कुठलंही software, कुठलीही रचना, कुठलाही “हे tech वापरू” चा प्रस्ताव: तीन प्रश्न विचारायची सवय लावा: “**इथे कुठला सौदा केलाय: काय विकून काय घेतलं?**” (Book 5), “**हा सौदा या परिस्थितीला योग्य आहे का, की fashion आहे?**” (Book 5), आणि “**हे मी agent ला बांधायला लावलं, तर दिशा आणि तपासणी माझी असेल का?**” (Book 4 + 5). हे तीन प्रश्न तुम्हाला रचना वापरणाऱ्यांकडून रचना ठरवणाऱ्यांकडे नेतील: आणि तिथेच पैसा, आणि स्वातंत्र्य, आहे.

## Chapter 4.7 [SPINE]: पाच हात-नकाशे (टेबलावर चिकटवा)

वाचायचं नाही, **वापरायचं**. पाच एक-पानी याद्या: रचना ठरवताना, database निवडताना, deploy करताना, नकाशा वाचताना, आणि agent-सोबत बांधताना समोर ठेवायच्या.

### नकाशा 1: नवं system: रचना-निर्णय (सुरुवातीला)

- [ ] सुरुवात सोपी: एक monolith, एक database; गरजेआधी तुकडे/scale नको (premature टाळा) (1.1, 2.6)
- [ ] डेटा: नाती+शुद्धता हवी? -> SQL (डिफॉल्ट). प्रचंड-सैल-वेगवान? -> NoSQL. शंका -> SQL (2.2)
- [ ] नाती आधी कागदावर (one-to-many, many-to-many) (2.1)
- [ ] कुठली माहिती किती जुनी चालेल? (cache/CAP निर्णय) (1.4, 1.10)
- [ ] सेवा वाढवल्या तर: फक्त API ने बोलतील, थेट DB-हात नाही (2.5)

### नकाशा 2: Scale ची भिंत आली (क्रम पाळा)

- [ ] आधी: index (जिथे शोध) + cache (वारंवारचं) (1.3, 1.4)
- [ ] मग: replicas कडे वाचनं वळवा (ताजं-हवं मुख्याकडे) (1.5)
- [ ] मग: load balancer + अनेक stateless servers (1.8)
- [ ] जड कामं queue त (idempotent!) (1.7)
- [ ] शेवटी, नाईलाजाने: sharding (लेखनच आवाक्याबाहेर तर) (1.6)
- [ ] "मोठा server" (vertical) हा शेवटचा/तात्पुरता उपाय

### नकाशा 3: अपयश-सज्जता + deploy (production आधी)

- [ ] प्रत्येक बाहेरची विनंती: timeout (वाट अनंत नाही) (1.11)
- [ ] retry: थांबत (backoff) + मोजून + idempotent (1.11)
- [ ] circuit breaker: आजारी दुवा वेगळा (1.11)
- [ ] deploy: canary (थोड्यांना आधी) + परत-फिरण्याचा रस्ता (3.3)
- [ ] छोटे-वारंवार deploys > मोठे-कचित (सुरक्षित) (3.3)
- [ ] backup आहे + तो तपासलेला आहे (GitLab!) (1.2, 2.9)

## नकाशा 4: नकाशा वाचणं (कुठलीही रचना, 2 मिनिटांत)

- [ ] एका विनंतीचा पूर्ण रस्ता काढा (कुठून कुठे) (1.15)
- [ ] एकच अपयश-बिंदू कुठे? (एक पडलं तर सगळं थांबतं) (1.8)
- [ ] database: पैसे SQL त? सैल NoSQL त? की सगळं कोंबलं? (2.2)
- [ ] cache कुठे? "जुनं कधी फेकायचं" विचारलंय का? (1.4)
- [ ] सेवा-संवाद: API ने की थेट-DB-हात (धोका)? (2.5)
- [ ] गर्दी/अपयश कुठे फुटेल? (queue? LB? timeout?) (1.11)
- [ ] प्रत्येक पेटी "का आहे"? उत्तर नसेल तर fashion (2.10)

## नकाशा 5: Agent-सोबत बांधणं (AI-युग)

- [ ] रचना-निर्णय मी घेतो (SQL/NoSQL, mono/micro, CAP):  
agent अंमलात आणतो (4.5)
- [ ] स्पष्ट करार आधी: काय + कुठले सौदे (Book 4, spec) (4.5)
- [ ] agent चं code: नकाशा-वाचन (नकाशा 4) लावून तपासा
- [ ] Book 4 ची शिस्त: test/evals + brakes + review + observ.
- [ ] over-engineering तपासा: agent ला "सोपं ठेव" सांगा  
(WhatsApp/Zerodha धडा) (4.1, 4.3)

**पाचही नकाशांमागचं एक वाक्य:** प्रत्येक निर्णयाला एकच प्रश्न: "इथे कुठला सौदा, आणि तो या परिस्थितीला योग्य आहे का?" Fashion ला नाही, सौद्याला हो. हीच पूर्ण Book 5 ची, आणि top engineer ची, एकमेव नजर.

# MASTER GLOSSARY: Book 5 चे सगळे रचना-शब्द

(शब्द -> एक ओळ -> chapter.)

|                   |                                                    |
|-------------------|----------------------------------------------------|
| ACID              | database ची शिस्त: अखंड-सुसंगत-एकांत-टिकाऊ (1.2)   |
| API               | दोन software मधला करार (menu-तत्त्व) (2.5)         |
| B-tree            | वाचन-जलद database-रचना (SQL) (2.3)                 |
| blameless         | चूक -> दोष नको, रचना का फसली (3.4)                 |
| blue-green        | जुनं चालू ठेवून नव्याकडे सुरक्षित वळण (3.3)        |
| cache             | वारंवारचं उत्तर जलद जागी; ताजेपणा विकून (1.4)      |
| canary            | नवं code आधी थोड्यांना (CrowdStrike-धडा) (3.3)     |
| CAP               | फूट-क्षणी C की A; तीनातले दोनच (1.10)              |
| CDN               | गिऱ्हाईकाजवळ static ची प्रत; वेग+bill (1.9)        |
| chaos engineering | मुद्दाम अपयश घडवून टिकाव तपासणं (4.4)              |
| CI/CD             | आपोआप build+test+deploy साखळी (3.3)                |
| circuit breaker   | आजारी दुवा वेगळा काढणारा fuse (1.11)               |
| code review       | दुसऱ्याची मोहोर; दर्जा+ज्ञान-प्रसार (3.1)          |
| deadlock          | दोन व्यवहार एकमेकांची वाट बघत अडकणं (2.4)          |
| egress            | cloud मधून data बाहेर काढण्याचं (महाग) भाडं (1.14) |
| feature flag      | नवं feature स्विचवर; बंद/चालू करता येतं (3.3)      |
| git               | इतिहास-यंत्र + branches + review (3.1)             |
| horizontal scale  | अनेक स्वस्त यंत्रं > एक महाग (1.8)                 |
| idempotency       | दोनदा केलं तरी परिणाम एकदाच (1.7)                  |
| index             | database ची अनुक्रमणिका; वाचन जलद (1.3)            |
| inverted index    | शब्द -> वस्तू; खऱ्या search चा पाया (2.7)          |
| load balancer     | विनंत्या मोकळ्या server कडे वळवणारा (1.8)          |
| LSM-tree          | लेखन-जलद database-रचना (NoSQL) (2.3)               |
| migration         | चालत्या database ची रचना सुरक्षित बदलणं (2.9)      |
| monolith          | पूर्ण app एकाच code मध्ये (2.6)                    |
| microservices     | app चे अनेक स्वतंत्र सेवा-तुकडे (2.6)              |
| normalization     | एक सत्य एका जागी; पुनरावृत्ती टाळणं (2.1)          |
| p99               | टोकाचं मोजमाप; सरासरीची फसवणूक टाळते (1.12)        |
| partition         | अनेक यंत्रांमधला संपर्क तुटणं (1.10)               |
| queue             | जड कामं रांगेत; धक्का शोषतो (1.7)                  |
| race condition    | एकाच वेळी दोघांच्या हातामुळे अनिश्चित निकाल (2.4)  |
| replication       | database ची प्रत; lag = किती जुनी (1.5)            |
| sharding          | database चे यंत्रांवर तुकडे; लेखन वाटलं (1.6)      |
| SQL/relational    | शिस्त+नाती+ACID database (2.2)                     |
| stateless         | server काहीही स्वतःकडे साठवत नाही (1.8)            |
| testing           | unit/integration/e2e थर; घसरण-रोध (3.2)            |
| timeout           | वाट किती बघायची, मर्यादा (1.11)                    |
| transaction       | अनेक पायऱ्या एक अभंग गड्डा (2.4)                   |
| vertical scale    | तेच यंत्र अधिक शक्तिशाली करणं (1.8)                |
| WebSocket         | सतत-उघडा दुवा; real-time साठी (2.8)                |



**पाचवें पुस्तक संपलं.** Machine जादू नाही (1); AI जादू नाही (2); जमीन जादू नाही (3); harness जादू नाही (4); आणि मोठं software जादू नाही (5): **सगळं सौदे आणि शिस्त आहे, आणि दोन्ही शिकता येतात.**

पाच पुस्तकांचा एकच सूर: **जादू शोधू नका; सौदे बघा.** गरज (1), यंत्र (1-2), विश्वास (4), रचना (5): प्रत्येक ठिकाणी तेच: काहीतरी विकून काहीतरी घ्यायचं, आणि जो तो सौदा **जाणून** करतो तोच जिंकतो.

आणि AI-युगात, जेव्हा बांधणं फुकट होतं, तेव्हा उरतं फक्त एक: **कुठला सौदा योग्य हे ठरवणारी नजर.** ती नजर आता तुमच्याकडे आहे. सुरू करा.